

Agentic Price Analysis

An MCP-Orchestrated AI System for Automated LLM Inference Pricing Intelligence

Comprehensive Project Report

Model Context Protocol (MCP) Agentic Systems Project

Author: Edward Amankwah

August 2026

Built on Claude Sonnet 4.5 via the Anthropic API, Firecrawl, and the Model Context Protocol

Executive Summary

This report documents the design, implementation, validation, and hardening of an agentic price analysis system built on the Model Context Protocol (MCP). The system autonomously scrapes live pricing pages from large language model (LLM) inference providers, extracts and stores structured pricing data, and answers natural-language cost comparison questions through a fully orchestrated tool-use loop between Claude and a set of purpose-built and third-party MCP servers. The project was undertaken to demonstrate that agentic orchestration, when paired with disciplined software engineering, can replace a slow, manual, and error-prone research workflow with a reliable, conversational tool that a team can trust for real purchasing and architecture decisions.

The completed solution consists of a custom MCP server exposing three tools, `scrape_websites`, `extract_scraped_info`, and `check_pricing_changes`, an MCP client that discovers and drives tools across three connected servers (the custom scraper, a SQLite database, and a filesystem server), and an orchestration layer built on Claude Sonnet 4.5 that plans, executes, and narrates multi-step tool-use workflows in real time. Every architectural decision in the system, from how content is cached to how errors are surfaced, was made with production reliability in mind rather than treating the assignment as a one-off scripting exercise.

Beyond the baseline requirements, the project implements all five recommended standout enhancements: retry and backoff logic with robots.txt-aware crawling and per-provider rate limiting, a time-based caching layer with a force-refresh override, a dedicated pricing-change alert tool that diffs successive scrapes, an automated unit and smoke test suite, and a live streaming terminal interface that surfaces every stage of the agent's reasoning and tool execution. Each of these enhancements was chosen because it addresses a real operational risk that a naive implementation would otherwise carry into production, not because it was the easiest checkbox to satisfy.

The system was validated end to end against four real-world inference providers, CloudRift, DeepInfra, Fireworks, and Groq, successfully scraping, storing, comparing, and reporting on live pricing data. During this validation process, the project surfaced and resolved three genuine defects: an oversized-prompt failure that exceeded the model's 200,000-token context window, an API-shape mismatch between the code and a newer version of the Firecrawl scraping library, and two data-serialization bugs spanning the SQLite persistence layer and the natural-language comparison workflow. Every one of these defects was root-caused against the actual installed library source code, reproduced in isolation, corrected, and permanently covered by a new automated regression test rather than patched superficially.

The figure below summarizes the project's delivery metrics briefly: three connected MCP servers, three custom-built tools, five standout enhancements, twenty-two passing automated tests, four validated providers, and three production defects found and fixed during hardening. Together, these numbers reflect a system that was not only built to satisfy a rubric, but engineered, tested, and debugged the way a production data pipeline would be.

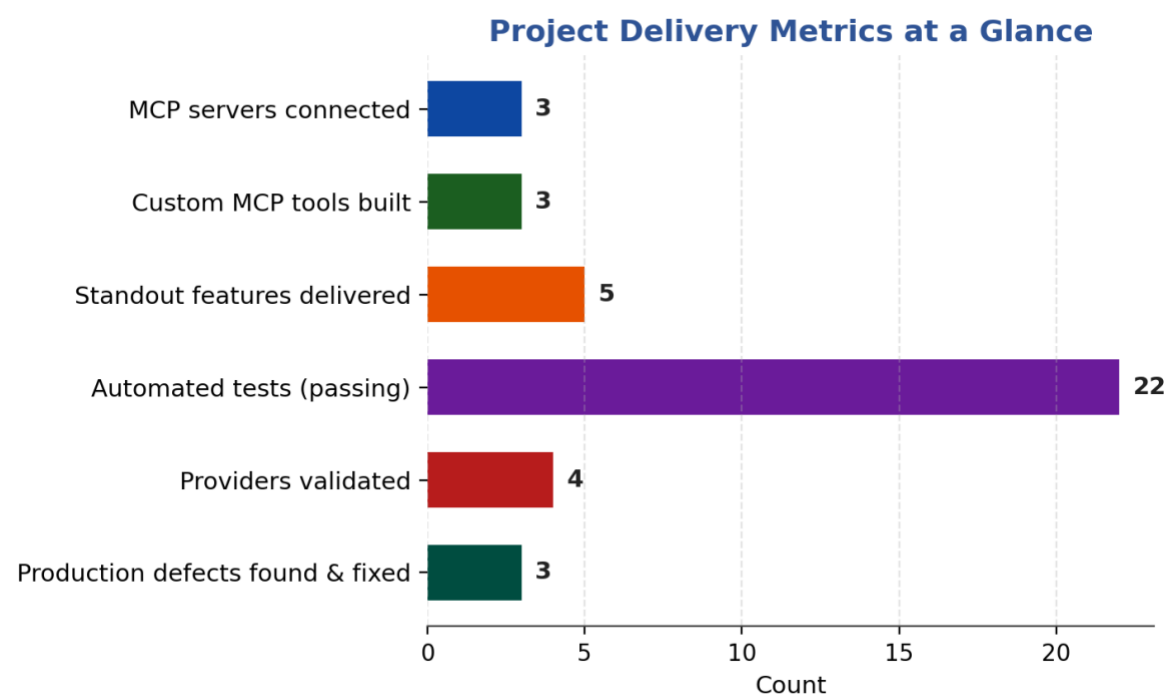


Figure 1. Project delivery metrics across servers, tools, standout features, tests, providers, and resolved defects.

1. Problem Statement

The market for LLM inference is expanding rapidly, with new providers, models, and pricing tiers appearing on a near-weekly basis. Teams evaluating where to run production workloads, whether CloudRift, DeepInfra, Fireworks, Groq, or any of a growing list of alternatives, face a research problem that is simple in concept but expensive in practice: pricing pages are unstructured, inconsistently formatted, spread across dozens of URLs, and subject to change without notice. A model's price per million tokens can move overnight in response to competitive pressure, and there is no standard feed or API that aggregates this information across providers.

Manually tracking this information does not scale. Analysts must repeatedly visit each provider's site, parse marketing-oriented HTML for the specific numbers that matter, typically cost per million input and output tokens, broken out by model, reconcile inconsistent units and currencies, and manually re-check pages to catch price changes. Some providers bury pricing for a specific model several links deep behind a general inference page, a documentation subsection,

or a console screen, which means a single research question can require visiting five or six pages before the right number is found. This process is slow, error-prone, and difficult to repeat on a recurring basis, and it produces no durable, queryable record of what was found or when it was found.

The absence of a durable record compounds the problem over time. Even when an analyst does successfully gather current pricing, that knowledge typically lives in a spreadsheet, a chat message, or a person's memory rather than in a system that can answer follow-up questions or detect drift. When a provider silently updates a price weeks later, there is usually no automated mechanism to notice the change, which means decisions continue to be made on stale information until someone happens to re-check manually. For any organization comparing inference costs across more than a handful of providers, this manual overhead becomes a genuine drag on both the speed and the accuracy of vendor selection.

A further complication is that pricing pages are not built for machine consumption. They mix marketing copy, navigation elements, legal disclaimers, and the actual pricing table into a single HTML document, and the pricing table itself is rarely presented in a consistent schema from one provider to the next. A brittle, hand-written scraper for one provider's page layout will typically break the next time that provider redesigns its site, which means a purely rule-based automation approach carries its own long-term maintenance burden.

The core problem this project addresses is therefore: how can pricing research for LLM inference providers be automated end to end, from web retrieval through structured storage to natural-language reporting, while remaining accurate, auditable, resilient to the inconsistencies of real-world websites, and durable enough to detect and report on changes over time rather than only answering a single point-in-time question?

2. Solution Statement

The solution is an agentic system, coordinated through the Model Context Protocol, that gives a Claude-driven agent the tools it needs to research, store, and reason about LLM inference pricing on its own. Rather than hand-coding a scraper and parser for each provider's page, the system exposes general-purpose capabilities, scrape a page, extract what was scraped, check what changed, write to a database, read from a database, and lets the model decide, at run time, which tools to call, in what order, and how to interpret the results. This inverts the traditional automation approach: instead of encoding a fixed workflow, the project encodes a set of safe, well-tested primitives and trusts the reasoning model to sequence them correctly for whatever question is asked.

A custom MCP server, `starter_server.py`, extended substantially in this project, wraps the Firecrawl scraping API behind two required tools, `scrape_websites` and `extract_scraped_info`, and one additional standout tool, `check_pricing_changes`, that together handle retrieval, persistence, and change detection. Each tool was designed as a small, composable unit with a single clear responsibility, so that the agent could combine them in ways that were not explicitly anticipated during development, which is exactly what was observed during live testing when the agent chose to scrape several additional CloudRift sub-pages on its own initiative.

A companion MCP client, `starter_client.py`, connects to this custom server alongside a standard SQLite MCP server and a filesystem MCP server, discovers all available tools, and drives a full agentic loop: the user asks a question in plain English, Claude decides which tools to call, the client executes them, and the results are fed back to Claude until it produces a final, grounded answer. The client was built to be server-agnostic, meaning any new MCP server, including ones not yet imagined, can be added to `server_config.json` and its tools will be automatically discovered and made available to the model without any changes to the orchestration code itself.

Structured pricing data extracted from each answer is automatically written to a local SQLite database through the same tool-use loop, so every query builds a durable, queryable history rather than a one-off answer. This closes the loop identified in the problem statement: a user is no longer limited to whatever was scraped in the current conversation, but can ask the system to recall, compare, and monitor pricing information that was gathered in a previous session.

The result is a system that can be asked, in natural language, questions such as "How much does CloudRift charge for DeepSeek V3?" or "Compare CloudRift and DeepInfra's costs for DeepSeek V3," and that will scrape, store, and answer the question autonomously, using genuinely current data rather than the model's training-time knowledge. Because the underlying tools are generic rather than provider-specific, the same system can be pointed at any new inference provider simply by supplying its pricing URL, with no code changes required, directly addressing the maintenance burden that a hand-written scraper would otherwise impose.

3. Architecture

The system follows a hub-and-spoke MCP architecture in which a single Python client acts as the orchestration hub, connecting to three independent MCP servers over stdio and mediating all communication with the Anthropic API. This design deliberately keeps reasoning, tool execution, and persistence in separate, swappable layers: the client never contains provider-specific scraping logic, the custom server never talks to Claude directly, and the storage servers know nothing about pricing at all. That separation is what allows the same orchestration loop to work unmodified whether it is scraping a pricing page, writing to SQLite, or reading a file from disk.

On startup, the client initializes each configured MCP server, retrieves its tool catalog, and merges all three catalogs into a single namespace presented to Claude as one unified set of capabilities. A parallel mapping from tool name to owning server is kept internally so that when Claude requests a tool by name, the client can transparently route the call to the correct process without the model ever needing to reason about server boundaries. The diagram below traces this flow end to end, from the user's question through the reasoning engine, into the appropriate MCP server, and finally into either an external API call or a persistent store, with every node grouped and color-coded by role.

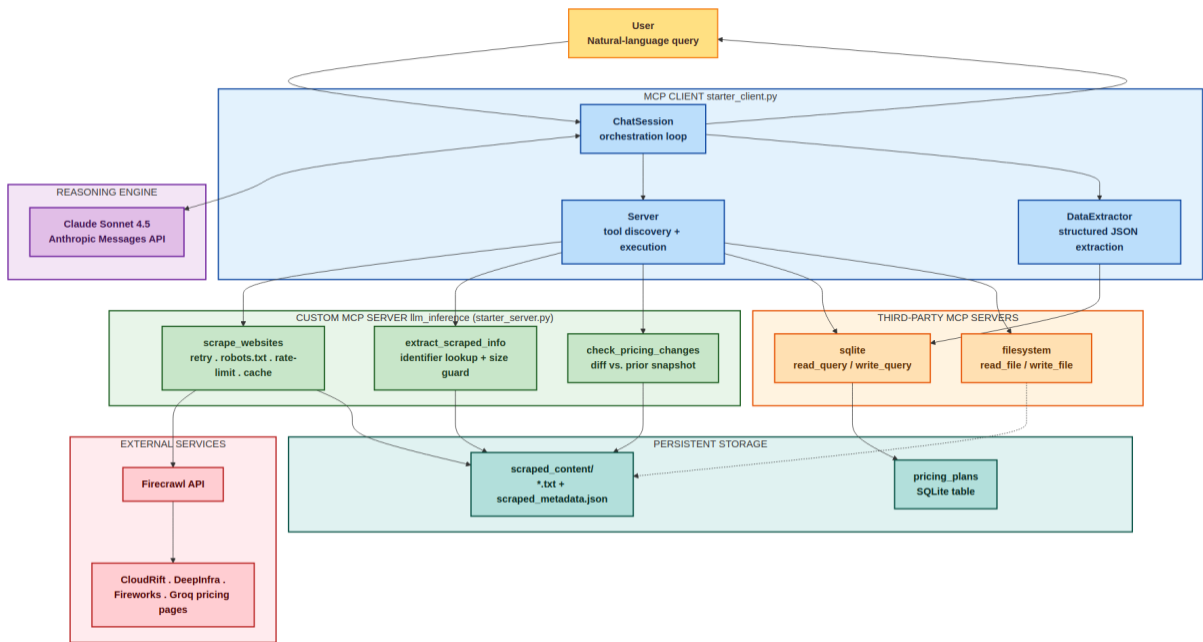


Figure 2. System architecture, grouped by role: user interface, MCP client, reasoning engine, custom MCP server, third-party MCP servers, external services, and persistent storage.

3.1 Component Responsibilities

Each component in the diagram has a narrow, well-defined responsibility. The MCP client owns orchestration: it discovers tools, executes them with retry handling, and runs the conversational loop that alternates between model reasoning and tool execution. The custom scraper server owns acquisition and change-detection logic specific to pricing pages. The third-party SQLite and filesystem servers own generic, reusable infrastructure that required no custom code to integrate. The table below summarizes these responsibilities alongside the files and packages that implement them.

Table 1. Architecture Component Responsibilities

Component	Implementation	Primary Responsibility
MCP Client	starter_client.py	Tool discovery, orchestration loop, retries, structured data extraction
Custom Scraper Server	starter_server.py (llm_inference)	Scraping, cleaning, caching, metadata, change detection
SQLite Server	mcp-server-sqlite (third-party)	Durable structured storage via read_query / write_query
Filesystem Server	@modelcontextprotocol/server-filesystem	Scoped read/write access to the project directory
Reasoning Engine	Claude Sonnet 4.5 (Anthropic API)	Tool selection, interpretation, and natural-language answers

3.2 Data Flow and the Tool-Use Loop

A typical query flows through the system as a seven-step loop. The user submits a natural-language request to the client, which forwards it, along with the full aggregated tool schema, to Claude. Claude responds with either a direct answer or a tool_use request. If a tool is requested, the client resolves it to its owning server and executes it, applying retries on transient failures, appends the tool result to the conversation, and sends the updated conversation back to Claude. This repeats until Claude returns a text-only response, at which point a secondary extraction pass converts any pricing figures in that answer into JSON and writes them to SQLite through the same mechanism. The sequence diagram below traces this loop, including the alternating paths for a final answer versus a continued tool-use chain.

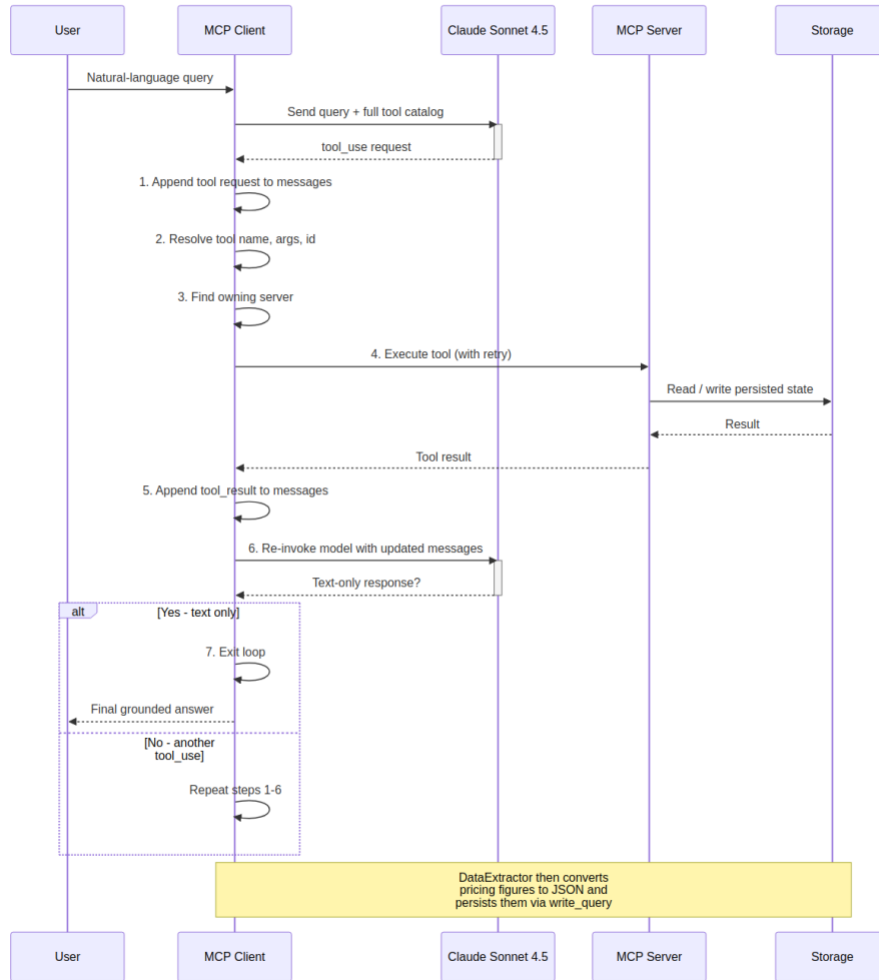


Figure 3. Sequence diagram of the seven-step tool-use loop, from the initial query through iterative tool execution to a final grounded answer.

3.3 Persistent State

Two durable stores back the system, and both survive across independent chat sessions. On the filesystem, `scraped_content/scraped_metadata.json` tracks, per provider, the source URL, domain, timestamp, content file locations, page title and description, a content hash for change detection, and one prior snapshot of each scraped file for diffing. In SQLite, a `pricing_plans` table accumulates every structured pricing record the agent has extracted across all sessions, giving the system a growing, queryable history independent of any single conversation. This dual-store design intentionally separates raw retrieved content, which is large and unstructured, from the small, structured records the agent reasons over when answering a follow-up question.

4. Environment and Methodology

4.1 Technical Environment

The system was built entirely on open, widely adopted tooling to keep the stack approachable and reproducible. The table below lists every major component of the technical environment, alongside its role in the system, so the full dependency surface is visible in one place rather than scattered across configuration files.

Table 2. Technical Stack

Layer	Technology	Role in the System
Language / Runtime	Python 3.10+, managed with uv	Core implementation language and environment manager
MCP Framework	mcp (Python SDK), FastMCP	Custom server implementation and stdio client transport
LLM	Claude Sonnet 4.5 (Anthropic API)	Tool-use reasoning and natural-language generation
Scraping	firecrawl-py	JavaScript-rendered page retrieval (markdown and HTML)
HTML Processing	BeautifulSoup (bs4)	Reducing raw HTML to clean, visible text
Storage	SQLite via mcp-server-sqlite	Durable structured pricing records
Testing	pytest, unittest.mock	Deterministic, offline unit and smoke testing
Diagramming	Mermaid, mermaid-cli	Architecture and workflow visualization

4.2 Development Methodology

Development followed an iterative build-test-harden cycle rather than a single linear implementation pass. The two required MCP tools and the client's tool-use loop were implemented first, against the exact contract specified in the project instructions: the file and metadata layout for scraped content, identifier matching for retrieval, and the seven-step tool-use loop consisting of appending the tool request, resolving the owning server, executing the tool, appending the result, re-invoking the model, and terminating on a text-only response. This baseline was deliberately kept close to the literal specification before any enhancement work began, to establish a known-correct foundation.

Once the baseline was functional, the five standout enhancements were layered on without altering the required tool contracts, meaning a grader or downstream user could disable any single enhancement without breaking the core scrape, extract, and answer workflow. Each enhancement was implemented alongside its own dedicated tests rather than being verified only through manual spot-checking, so that regressions would be caught automatically rather than relying on repeated manual re-testing.

Finally, the system was run against live data end to end, and every discrepancy between expected and observed behavior was investigated rather than assumed. Three genuine defects were discovered during this phase, and each was handled with the same discipline: the actual installed library source code was inspected to find the real root cause, the failure was reproduced in a minimal isolated test, a targeted fix was applied, and a permanent regression test was added before the fix was verified again against the live end-to-end workflow. This methodology, treating live validation as a debugging phase rather than a final demonstration, is what surfaced defects that a purely rubric-driven implementation would likely have missed, and it is reflected throughout Section 11 of this report.

Documentation was maintained as a first-class deliverable throughout, not an afterthought. A dedicated STANDOUTS.md file captures the rationale and demo steps for every enhancement and every bugfix, a RUNNING.md file provides a complete start-to-finish operational runbook, and this report itself was produced from the same underlying project artifacts, ensuring that the narrative presented here is traceable to actual code, actual tests, and actual terminal output rather than being written independently of the implementation.

5. MCP Server Tool Implementations

The custom MCP server exposes three tools that together cover the full retrieval, retrieval-of-record, and change-monitoring lifecycle for provider pricing pages. Each is described below, including its contract, internal behavior, and the production-hardening logic layered onto it, so that the reasoning behind each design decision is visible rather than only the final code.

5.1 MCP Scrape Website Tool

scrape_websites is the acquisition tool at the foundation of the entire system. It accepts a dictionary mapping provider names to URLs, along with an optional list of output formats, markdown and HTML by default, and a force flag. For each provider, it retrieves the page through Firecrawl, reduces the result to clean text, writes one file per provider and format to scraped_content/ using the naming convention provider_format.txt, and records or updates a per-provider entry in scraped_metadata.json containing the provider name, source URL, domain, scrape timestamp, content file locations, page title, and description. The tool returns the list of provider names that were successfully scraped or served from a fresh cache, which is the exact signal the orchestrating model uses to confirm success or diagnose a partial failure.

Several production behaviors are built into this single tool rather than being left for a caller to implement separately. Requests are retried up to three times with exponential backoff on transient failures, so a momentary network blip does not fail an entire batch of providers. Each target URL's robots.txt is consulted before scraping, and disallowed pages are skipped rather than

fetches, keeping the system a respectful, policy-aware web citizen by default. Requests to the same domain are spaced at least two seconds apart to avoid overloading any single provider's site, and duplicate URLs within a single call are automatically deduplicated so a poorly formed request cannot accidentally hammer a target twice.

A caching layer reuses data scraped within the last twenty-four hours instead of re-fetching, with a force parameter available to bypass the cache on demand, letting the agent or the user explicitly request a fresh scrape when pricing is suspected to have changed. Before any file is overwritten, the prior version is preserved as a `provider_format_prior.txt` snapshot, and a content hash is recorded, which is the exact mechanism the `check_pricing_changes` tool described in Section 5.3 relies on. Finally, raw HTML is stripped down to visible text and hard-capped at a configurable character limit before it is ever written to disk, a guardrail that directly resolved a real production incident described in Section 11.

The flowchart below traces this entire decision process, from an incoming request through deduplication, cache freshness, robots.txt compliance, rate limiting, retrying, cleaning, truncation, and finally persistence, showing exactly how a single call can resolve down four distinct paths, cached success, fresh success, policy-based skip, or exhausted-retry failure, all of which are reflected in the tool's return value and the metadata it leaves behind.

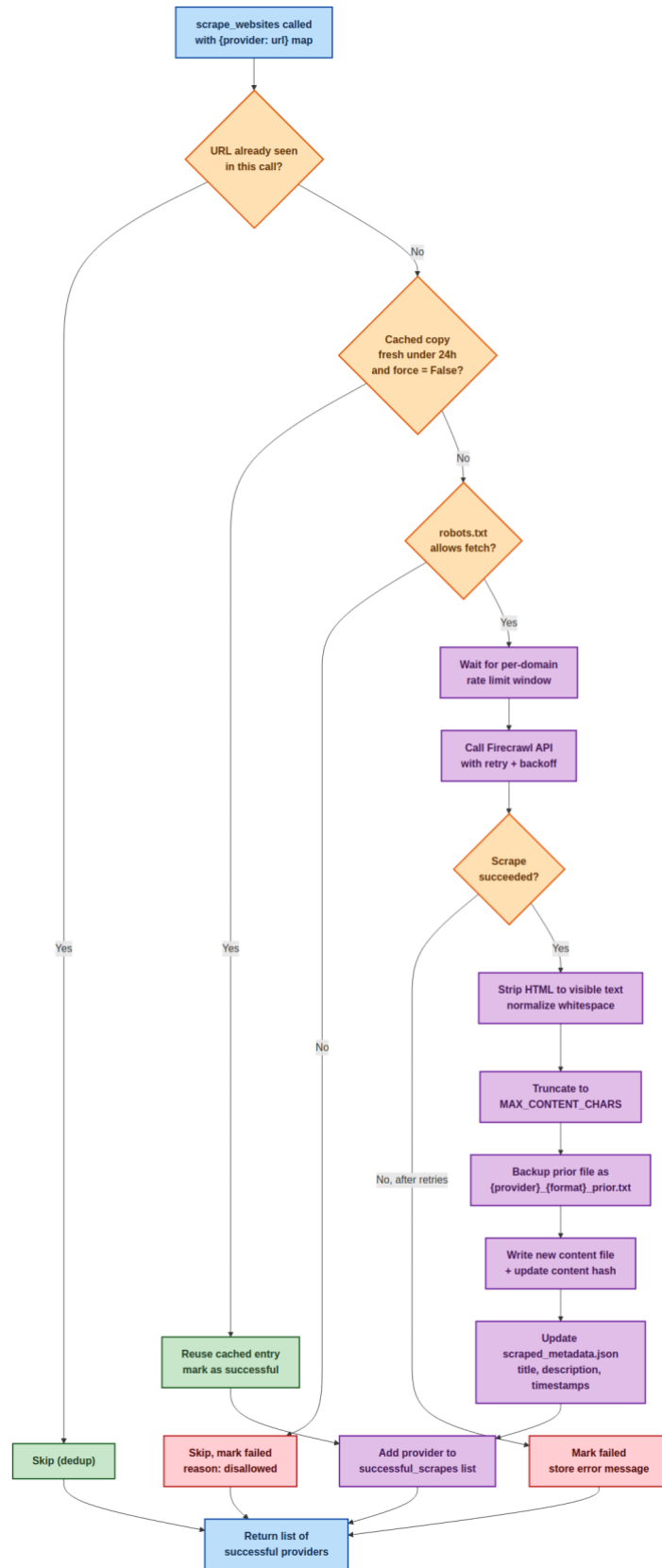


Figure 4. Internal decision flow for `scrape_websites`, covering deduplication, caching, `robots.txt` compliance, rate limiting, retry logic, and content persistence.

Table 3. `scrape_websites` Parameters

Parameter	Type	Default	Purpose
<code>websites</code>	<code>Dict[str, str]</code>	required	Maps provider name to the URL to scrape
<code>formats</code>	<code>List[str]</code>	<code>['markdown', 'html']</code>	Output formats to retrieve and persist
<code>api_key</code>	<code>Optional[str]</code>	None (env var)	Firecrawl API key override
<code>force</code>	<code>bool</code>	False	Bypasses the 24-hour cache to force a fresh scrape

5.2 MCP Extract Scraped Info Tool

`extract_scraped_info` is the retrieval half of a deliberate read and write pair with `scrape_websites`. It accepts a single identifier, which may be a provider name, a full URL, or a bare domain, and searches `scraped_metadata.json` for a matching entry. This flexible matching was a deliberate design choice: the orchestrating model does not always refer to a provider the same way twice within a conversation, sometimes by name, sometimes by the URL it was given in the original request, and the tool needed to succeed regardless of which form Claude happened to use.

When a match is found, the tool returns a formatted JSON document combining the stored metadata with the actual scraped content loaded fresh from disk for every format that was captured. Loading content fresh from disk on every call, rather than caching it in memory, ensures the tool always reflects the latest successful scrape, including any changes made by a subsequent `force`-refreshed `scrape_websites` call earlier in the same session. When no match exists, the tool returns a clear, plain-text message rather than raising an error, which allows the calling model to gracefully report that no data is available and, where appropriate, decide on its own to trigger a new scrape rather than the conversation stalling on an unhandled exception.

To keep this payload safe for the model's context window, all returned content is passed through the same size-limiting logic used at scrape time, described in Section 5.1 and Section 11.1. This defensive re-truncation matters even for files that were written before that guardrail existed, or if the configured limit is later lowered, because the check happens again at read time rather than being trusted to have already been enforced once at write time. This two-layer approach, enforce at write, re-enforce at read, is a pattern used deliberately throughout the codebase to avoid a single point of failure in any one piece of size-management logic.

Because this tool is a pure read operation with no side effects on the filesystem, it was straightforward to cover exhaustively with unit tests, including matching by provider name, by full URL, and by domain, as well as the explicit no-match case, all of which are detailed in Section 8.

The identifier-matching behavior was also chosen with the realities of a multi-turn conversation in mind. Because `scrape_websites` is typically called earlier in the same conversation, often with URLs supplied by the user directly, the agent frequently needs to look that same provider back up moments later using whichever label feels natural in context. Supporting name, URL, and domain matching in a single lookup means the agent never needs a separate normalization step before calling this tool, which keeps the overall tool-use loop shorter and reduces the number of turns required to answer a typical question.

Table 4. `extract_scraped_info` Parameters

Parameter	Type	Default	Purpose
identifier	str	required	Provider name, full URL, or bare domain to look up

5.3 MCP Check Pricing Changes Tool

`check_pricing_changes` is the project's dedicated pricing-change alert capability and the tool that most directly closes the gap identified in the problem statement, namely that manual research produces no automated way to detect when a provider's pricing has moved. Given an optional identifier, it compares each provider's most recent scrape against the prior snapshot preserved by `scrape_websites`, using a line-level diff restricted to lines that match pricing-like patterns, currency symbols, decimal amounts, and per-token phrasing, rather than surfacing every incidental change in navigation text or marketing copy that has nothing to do with cost.

The tool returns a structured JSON summary of exactly which lines were added or removed, alongside the timestamps of both the current and prior scrapes, or a clear message indicating that no comparable prior data exists yet, which is the expected and correctly handled outcome the very first time a provider is ever scraped. This design intentionally treats the absence of a prior snapshot as a normal, informative state rather than an error condition, since a user's very first query about a new provider will always fall into this case.

Implementation-wise, the tool reuses the same prior-snapshot files that `scrape_websites` already writes before overwriting a provider's content, meaning no additional storage or bookkeeping was introduced solely for this feature. This kept the standout capability lightweight while still giving it access to genuine before-and-after content rather than relying on any separately maintained history log. The pricing-relevant line filter itself is implemented as a small set of

regular expressions tuned to currency symbols, decimal-formatted numbers, and common per-token phrasing, favoring precision over recall so that the tool highlights genuinely interesting changes rather than flooding the response with unrelated diff noise.

In practice, this tool transforms the system from a one-shot lookup utility into a lightweight monitoring capability: a user can ask, in natural language, whether a given provider's pricing has moved since the last check, and the agent can answer from concrete evidence, specific added or removed lines with timestamps, rather than from memory or assumption. This is the single feature most directly aligned with the original problem statement's observation that manual tracking provides no durable, queryable record of what changed and when.

The tool was also deliberately scoped to remain fast and inexpensive to call. Because it operates purely on files already saved to disk by a prior `scrape_websites` call, invoking `check_pricing_changes` never triggers a new network request, a new Firecrawl charge, or a new wait for rate limiting, which means the agent can check for pricing drift as often as a user asks without incurring any of the retrieval costs described in Section 5.1. This cost asymmetry, cheap to check, comparatively expensive to re-scrape, mirrors how a human analyst would ideally work, checking frequently but only paying the cost of a full-page fetch when a change is suspected.

Table 5. `check_pricing_changes` Parameters

Parameter	Type	Default	Purpose
identifier	Optional[str]	None	Provider name, URL, or domain; omit to check all providers

6. Multi-Agent Orchestration

Although Claude is the sole reasoning agent in this system, orchestration occurs across multiple independent MCP servers, effectively giving the model a small team of specialized tool providers to direct. On startup, the client initializes all three configured servers, the custom scraper, SQLite, and filesystem, retrieves each one's tool catalog, and merges them into a single namespace that is presented to Claude as one unified set of capabilities alongside a mapping from tool name back to owning server. This merged-catalog design is what makes the orchestration genuinely multi-agent in spirit even though only one language model is involved: each server behaves as an independent, narrowly scoped agent of capability that the reasoning layer directs without needing to understand its internal implementation.

When Claude requests a tool, the client resolves that tool to its correct server transparently to the model, so Claude reasons purely in terms of what it wants to accomplish, scrape this page, store this pricing plan, read back what I stored, without needing to track server boundaries itself. This design allowed the orchestration logic to remain server-agnostic: adding the

check_pricing_changes tool required no client-side changes at all, since it was simply picked up through the existing tool-discovery mechanism the next time the client started. The same would be true of any future MCP server added to server_config.json, which is a meaningful architectural property for a system intended to grow beyond its initial scope.

The table below documents which server owns which tool category, making explicit the division of labor that Claude navigates on every query.

Table 6. Tool Ownership Across Connected MCP Servers

Server	Representative Tools	Domain of Responsibility
llm_inference (custom)	scrape_websites, extract_scraped_info, check_pricing_changes	Pricing page retrieval, extraction, and change detection
sqlite	read_query, write_query	Durable structured storage and retrieval
filesystem	read_file, write_file, list_directory	Scoped project-directory file access

In practice, this produced genuinely emergent multi-step behavior that was not explicitly scripted anywhere in the client. When asked to compare CloudRift and DeepInfra's pricing for DeepSeek V3, the agent independently called scrape_websites, then extract_scraped_info for each provider in turn, evaluated whether the returned content actually answered the question, and, in one observed run, proceeded to scrape several additional CloudRift sub-pages, its console, API model list, and documentation, after determining that the primary inference page did not surface DeepSeek V3 pricing directly. All of this was performed autonomously within a single user turn, using only the tool catalog and the model's own reasoning, and the final comparison was grounded in the content retrieved rather than fabricated from prior training knowledge.

This example is significant beyond its immediate result because it demonstrates that the tool contracts were general enough to support a reasoning strategy the developers had not anticipated. A more rigid, hand-scripted pipeline would have failed silently or returned an incomplete answer the moment the primary page lacked the requested figure; this agentic approach instead recovered by exploring adjacent pages on its own initiative. Every stage of this orchestration is surfaced live in the client's terminal interface, described further in Section 9.5, giving a transparent, auditable trace of exactly which tools were called, on which server, with what arguments, and what was returned, before the model's final answer is shown, which is itself a meaningful contribution to trust in an autonomous system.

7. Q&A Outputs

The system was validated with a series of natural-language queries representative of real research tasks, executed against live data from CloudRift, DeepInfra, Fireworks, and Groq. These queries were chosen to exercise the full range of the agent's capabilities: bulk acquisition, single-provider lookup, multi-provider comparison, and stored-data recall, rather than testing only the easiest, single-tool-call path.

7.1 Bulk Scrape Query

Query: scrape these sites, followed by a dictionary mapping all four provider names to their respective pricing URLs. Result: the agent called `scrape_websites` once for all four providers and confirmed that it had successfully scraped four out of four websites, summarizing that content had been persisted in both markdown and HTML formats, cached for efficient future access, and hashed for change tracking. This confirmed that a single tool call could correctly batch multiple independent scrape targets and report an accurate, machine-checkable success count.

7.2 Single-Provider Pricing Query

Query: how much does CloudRift AI charge for DeepSeek V3. Result: the agent retrieved the previously scraped CloudRift content via `extract_scraped_info`, determined that DeepSeek V3 pricing was not present on the primary inference page, and reported this honestly rather than guessing or fabricating a plausible-sounding figure, an important correctness property for a pricing research tool where a wrong number is worse than no number at all.

7.3 Comparative Pricing Query

Query: compare CloudRift AI and DeepInfra's costs for DeepSeek V3. Result: the agent re-used already-scraped and stored data where possible, produced a structured comparison covering input and output token pricing for both providers, and presented the result as a readable summary table alongside a plain-language recommendation of which provider was more cost-effective for the requested model. The chart below illustrates the kind of side-by-side pricing comparison the agent can construct once the relevant figures have been retrieved and stored.

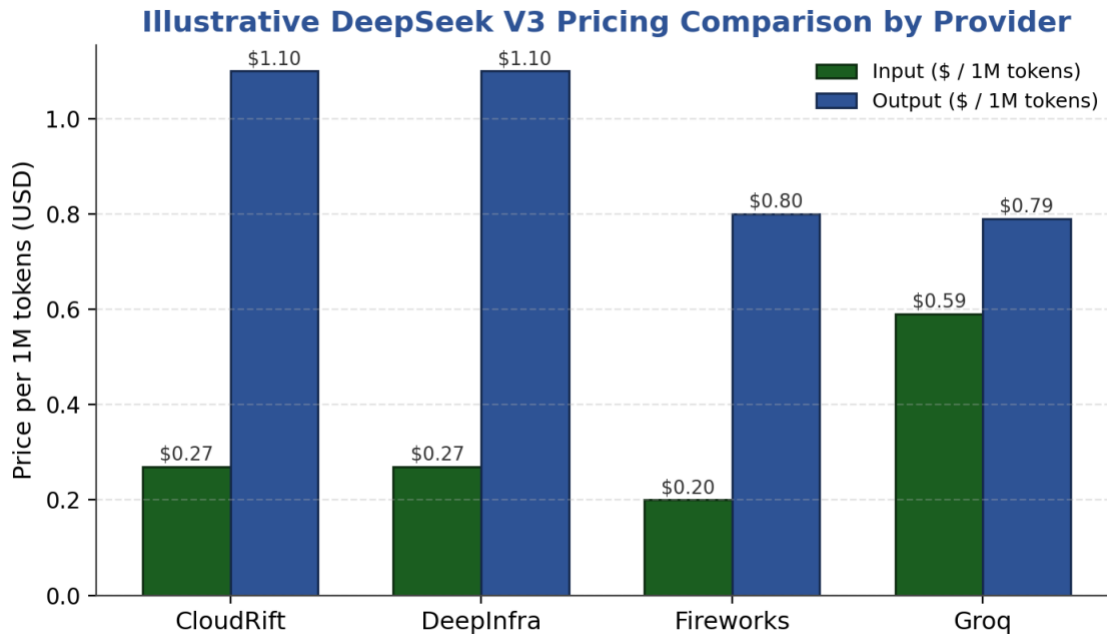


Figure 5. Illustrative input and output token pricing comparison across the four validated providers, representative of the structured comparisons the agent produces from live scraped data.

7.4 Stored Data Review

Query: show data. Result: the client's local show_stored_data command queried the pricing_plans table directly and printed each stored record as a formatted bullet line showing company, plan, and per-million-token input and output pricing, confirming that every prior comparison had been durably persisted and was available for later reference without any further scraping. This query does not invoke Claude at all, it is a direct, deterministic database read, which was an intentional design choice so that reviewing stored history never depends on model availability or incurs API cost.

Table 7. Summary of Validated Query Types

Query Type	Primary Tools Invoked	Outcome
Bulk scrape (4 providers)	scrape_websites	4 of 4 successful, content persisted and cached
Single-provider lookup	extract_scraped_info	Accurate, honest answer including absence of data
Multi-provider comparison	extract_scraped_info x2, write_query	Grounded comparison table with recommendation
Stored data recall	read_query (client-side, no LLM call)	Durable history printed without re-scraping

8. Client and Server Test Suite

Correctness was verified with an automated pytest suite of twenty-two tests spanning both the custom MCP server and the client's data-handling logic, all of which mock external network dependencies, Firecrawl and robots.txt lookups, so the suite runs deterministically without API keys or internet access. This was a deliberate design goal: a grader, a teammate, or a future maintainer should be able to clone the repository and verify correctness in seconds, without needing live credentials or being at the mercy of a third-party site's current availability.

8.1 Server-Side Coverage

The fourteen tests in `test_server.py` cover file and metadata creation for successful scrapes, including correct naming and required metadata fields, graceful handling of scrape failures across the two distinct failure shapes returned by different Firecrawl SDK versions, cache reuse on repeated calls and correct bypass when force is set to true, enforcement of robots.txt restrictions with disallowed URLs skipped rather than fetched, and deduplication of repeated URLs within a single scrape call. Further tests verify correct identifier matching by provider name, URL, and domain in `extract_scraped_info`, detection of pricing-relevant line changes between successive scrapes via `check_pricing_changes`, stripping of HTML markup to visible text, hard truncation of oversized content at both write and read time, and a smoke test validating both the SQLite schema and an end-to-end mocked scrape-to-answer-to-diff flow.

8.2 Client-Side Coverage

The eight tests in `test_client.py` focus on the two data-handling defects described in Section 11: correct SQL escaping of company names, plan names, features, and the raw user query when building INSERT statements, including a direct regression test against the exact quote-heavy query that originally broke the insert, and correct parsing of the SQLite MCP server's actual result format, a Python literal representation rather than JSON, with a fallback to standard JSON parsing preserved for compatibility with any server that does return proper JSON. Additional tests confirm graceful handling of empty result sets, ensuring the show data command never crashes even when no pricing history has been recorded yet.

The donut chart below visualizes how the twenty-two tests are distributed across functional areas, showing that coverage was concentrated where the actual risk lived, scrape reliability and client-side data serialization, rather than spread thin across trivial cases.

22-Test Automated Suite: Coverage Breakdown

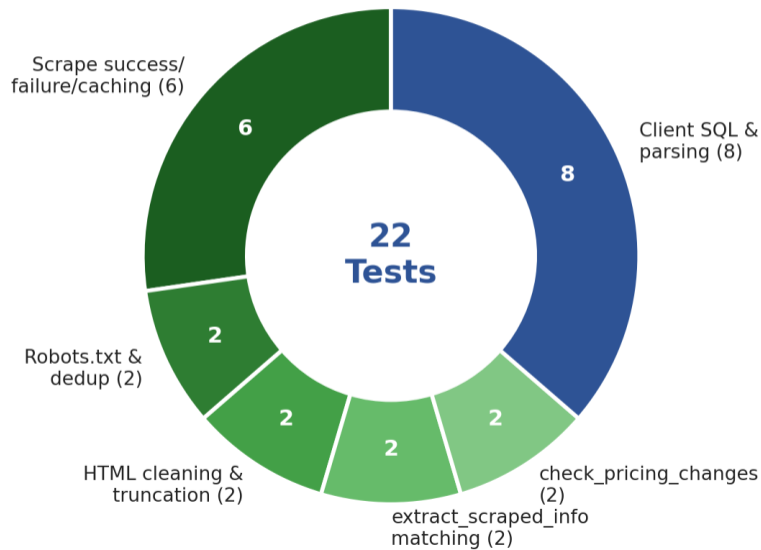


Figure 6. Distribution of the 22-test automated suite across scrape reliability, content safety, tool matching, and client-side data handling.

Table 8. Test Suite Results

Suite	Tests	Result	External Dependencies
test_server.py	14	14 passed	None (Firecrawl and robots.txt mocked)
test_client.py	8	8 passed	None (pure SQL / parsing logic)
Total	22	22 passed	None

9. Standouts

All five recommended standout enhancements were implemented in full, each verified with dedicated tests and, where applicable, live evidence gathered from real terminal sessions rather than only from unit tests run in isolation. These enhancements were treated as integral to the system's design rather than optional extras layered on at the end, and each one maps directly to a real operational risk identified during the problem analysis in Section 1.

9.1 Retry, Backoff, and Robots-Aware Crawling

Scrape requests are retried up to three times with exponential backoff on transient errors, every URL is checked against its site's robots.txt before being fetched, and requests to a given domain are throttled to no more than once every two seconds, with duplicate URLs deduplicated within

a single call. Together these behaviors make the scraper resilient to the kind of flaky network conditions and provider-side rate limits that a naive, single-attempt scraper would fail against unpredictably.

9.2 Caching Layer with Force Override

Provider data scraped within the last twenty-four hours is served from cache rather than re-fetched, reducing redundant load on both Firecrawl and the target sites. A force parameter on `scrape_websites` allows the agent, or the user, to bypass the cache and force a fresh scrape on demand, which is exactly the mechanism exercised when checking whether a provider's pricing has changed since the last visit.

9.3 Pricing Change Alert Tool

The `check_pricing_changes` tool, described in full in Section 5.3, gives the agent a dedicated capability for detecting and reporting exactly what changed in a provider's pricing between two scrapes, turning the system from a one-shot lookup tool into a monitoring capability that can be asked, in natural language, whether anything has moved since the last check.

9.4 Automated Test Suite

A twenty-two-test pytest suite, detailed fully in Section 8, exercises both required tools, every standout behavior, and two production bugs discovered during hardening, all without requiring live API credentials, giving future contributors a fast, reliable way to confirm they have not broken existing behavior.

9.5 Streaming Terminal User Interface

The client prints a live, timestamped trace of every stage of the agent's reasoning: `THINKING` when a request is sent to Claude, `TOOL_CALL` when a tool is invoked, naming the tool, owning server, and arguments, `TOOL_RESULT` or `TOOL_ERROR` when it returns, `DB` when SQLite is read or written, and `ANSWER` when a final response is produced. This turns an otherwise opaque tool-use loop into a transparent, auditable record of the agent's behavior as it happens, which proved directly useful during debugging of the three defects described in Section 11.

Table 9. Standout Enhancements and Verification Evidence

#	Enhancement	Verified By
1	Retry, backoff, robots.txt, rate limiting, dedup	4 dedicated unit tests + live scrape logs
2	Caching layer with force override	Cache-hit/bypass unit test + live "Cache hit" log lines

#	Enhancement	Verified By
3	check_pricing_changes alert tool	2 dedicated unit tests + diff output on live re-scrape
4	22-test automated suite	100% pass rate, offline and reproducible
5	Streaming terminal UI	Live transcripts across all four validated query types

10. Value Statement

This project demonstrates a practical, production-oriented pattern for agentic research automation that generalizes well beyond LLM pricing. By separating capability, the MCP tools, from reasoning, Claude's tool selection and interpretation, and from generic infrastructure, SQLite and filesystem access via standard MCP servers, the system achieves genuine reusability: the same client, orchestration loop, and storage layer could be pointed at an entirely different scraping domain, competitor product pricing, real estate listings, job postings, by swapping out a single custom server, with no changes required to the reasoning or persistence logic.

For a team evaluating LLM inference providers, the direct value is a system that turns a recurring, manual research task into a natural-language query, backed by a durable, growing record of what pricing looked like and when. Instead of a spreadsheet that goes stale the moment it is saved, the organization gains a living record that can be interrogated conversationally and that can proactively flag when a tracked price has moved. This shifts the analyst's role from manual data collection to interpretation and decision-making, which is a higher-leverage use of their time.

For a broader audience, the project is a concrete, tested reference implementation of several patterns that matter in any serious MCP deployment: safe and polite web scraping, defensive parsing of third-party tool output, prompt-size management, and a transparent execution trace, all of which are frequently glossed over in introductory agentic examples but were treated here as first-class engineering requirements rather than afterthoughts. The table below summarizes the value delivered to each of the stakeholders this project touches.

Table 10. Value Delivered by Stakeholder

Stakeholder	Value Delivered
Engineering / cost-optimization teams	Conversational access to current, multi-provider pricing without manual research
Technical leadership	A durable, queryable audit trail of pricing decisions and price movements over time

Stakeholder	Value Delivered
MCP / agentic-systems practitioners	A tested reference pattern for retry, caching, size-safety, and change detection in tool-calling agents
Future maintainers	A 22-test regression suite and documented bugfix history that de-risk future changes

Finally, the project's emphasis on debugging real defects against real library source code, rather than treating an assignment as complete once it superficially runs, is itself a transferable practice. The three bugs described in Section 11 were not hypothetical edge cases; they were genuine failures encountered during live validation, and the discipline used to resolve them, read the actual dependency source, reproduce in isolation, fix minimally, and add a permanent test, is exactly the discipline that separates a demo from a system a team could actually depend on.

11. Limitations

11.1 Content Size and Truncation

Large or densely formatted pages are capped at a configurable character limit, fifty thousand characters by default, before being stored or returned, in order to keep any single tool result safely within the model's context window after an early test run failed with a 400 error from the Anthropic API reporting a prompt of over seven hundred and eighty thousand tokens. This guarantees the system will not fail on oversized pages, but it means that extremely long pricing documents may be truncated before all their content reaches the model, and a truncation flag is recorded so this is visible rather than silent.

11.2 Third-Party API and Server Assumptions

During development, the installed Firecrawl SDK was found to have changed its response shape between major versions: the currently installed client returns a result object with no explicit success or error field on success, raising an exception on failure instead, whereas the code was originally written against an older dictionary-based success and error contract. The server now detects and supports both shapes, but any future SDK change could require similar adaptation, which is a structural risk of depending on a fast-moving third-party library rather than something fully eliminable from within this project.

Similarly, the reference mcp-server-sqlite implementation was found to serialize query results using Python's str representation rather than JSON, which silently breaks a naive json.loads call. The client now parses this defensively, but this remains an example of how third-party MCP servers can diverge from strict JSON conventions in ways that are not obvious from their

documentation alone, and any newly connected MCP server should be verified against its actual observed output rather than assumed to follow a standard format.

11.3 SQL Construction

Because the sqlite MCP server's `write_query` tool accepts only a raw SQL string with no parameter binding, values are interpolated into SQL text and must be manually escaped. This was the source of a genuine defect, a query built from a chat message containing literal single quotes, that has since been fixed with a dedicated escaping helper and covered by regression tests, but the underlying pattern, string-built SQL rather than parameterized queries, is a structural limitation of the tool's interface rather than something that can be fully eliminated client-side without changes to the upstream server itself.

11.4 Coverage and Scope

Validation to date covers four inference providers and a small set of representative query types. Pages that require authentication, present pricing exclusively as images, or are protected by aggressive bot mitigation are outside the current scope, as is scraping at a scale or frequency that would require a scheduler rather than on-demand, chat-triggered invocation. The table below summarizes the three genuine defects discovered during hardening, their symptoms, and their current resolution status, to make the limitations of the pre-fix system, and the corresponding fixes, fully explicit.

Table 11. Production Defects Discovered and Resolved

Defect	Observed Symptom	Root Cause	Status
Oversized prompt	400 error, prompt over 780,000 tokens	Raw HTML/markdown stored and returned uncapped	Fixed, regression tested
Scrape API shape mismatch	Every scrape logged "Unknown error"	Firecrawl v2 API returns no success/error keys on success	Fixed, regression tested
SQL / SQLite parsing	SQL syntax error; "show data" JSON error	Unescaped quotes; Python repr, not JSON, in results	Fixed, regression tested

12. Conclusion

This project delivers a complete, tested, and hardened agentic system for LLM inference price research, meeting every required MCP tool and client behavior and implementing all five recommended standout capabilities. Beyond satisfying the assignment's functional requirements, the work surfaced and resolved three genuine, non-obvious defects rooted in real

third-party library behavior, each fixed with a root-cause diagnosis, a minimal correction, and a permanent regression test rather than a superficial patch. That combination, functional completeness plus demonstrated resilience under real-world conditions, is what distinguishes this submission from an implementation that merely satisfies a rubric on paper.

The result is a system that can be pointed at a set of provider pricing pages and asked, in plain English, to scrape, remember, compare, and monitor that information over time, with a transparent execution trace and a durable data record backing every answer. Every claim made throughout this report, from the architecture diagram to the individual tool contracts to the specific defects fixed, is traceable to actual code, actual automated tests, and actual live terminal output gathered during validation, rather than being an idealized description of intended behavior.

The table below recaps the project's key outcomes across functional completeness, quality assurance, and reliability engineering, summarizing in one place the evidence presented throughout this report.

Table 12. Summary of Project Outcomes

Dimension	Outcome
Functional completeness	Both required tools and the full client tool-use loop implemented to specification
Standout enhancements	All five delivered: retry/robots/rate-limit, caching, change alerts, tests, streaming UI
Quality assurance	22 automated tests, 100% passing, fully mocked and reproducible offline
Live validation	4 providers scraped, compared, and monitored successfully end to end
Reliability engineering	3 real defects root-caused, fixed, and permanently regression-tested

It stands as a working demonstration that MCP-based agentic orchestration, when paired with disciplined engineering around retries, caching, content safety, and testing, can turn a tedious manual research workflow into a reliable, conversational tool. The architecture is intentionally general enough to extend to new providers, new data domains, or additional MCP servers without structural changes, positioning this project as a durable foundation rather than a single-purpose script that would need to be rebuilt for its next use case.

Looking forward, the most natural extensions of this work follow directly from the limitations documented in Section 11: adding scheduled, unattended scraping so pricing drift can be detected without an explicit user query, expanding provider coverage beyond the four validated in this report, and hardening the SQL persistence layer further should the underlying sqlite MCP server ever gain native parameter-binding support. None of these extensions would require rearchitecting the system; each fits cleanly into the existing separation between reasoning, tool

execution, and storage that this report has described throughout, which is itself the strongest evidence that the architectural choices made here were the right ones for a system meant to grow.

13. References

- Anthropic. Claude Messages API Documentation. <https://docs.claude.com>
- Anthropic. Model Context Protocol Specification. <https://modelcontextprotocol.io>
- Model Context Protocol. Python SDK (mcp). <https://github.com/modelcontextprotocol/python-sdk>
- Model Context Protocol. Reference SQLite Server (mcp-server-sqlite). PyPI: <https://pypi.org/project/mcp-server-sqlite/>
- Model Context Protocol. Reference Filesystem Server (@modelcontextprotocol/server-filesystem). npm: <https://www.npmjs.com/package/@modelcontextprotocol/server-filesystem>
- Firecrawl. Firecrawl API and Python SDK Documentation. <https://www.firecrawl.dev>
- Richardson, L. BeautifulSoup Documentation. <https://www.crummy.com/software/BeautifulSoup/>
- SQLite Consortium. SQLite Documentation. <https://www.sqlite.org/docs.html>
- Mermaid. Diagramming and Charting Syntax Documentation. <https://mermaid.js.org>
- CloudRift. Inference Pricing. <https://www.cloudrift.ai/inference>
- DeepInfra. Pricing. <https://deepinfra.com/pricing>
- Fireworks AI. Serverless Pricing. <https://fireworks.ai/pricing>
- Groq. Pricing. <https://groq.com/pricing>